上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



嵌入式软件测试

上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



第一部分：理解嵌入式系统-实时特性

Computing Systems

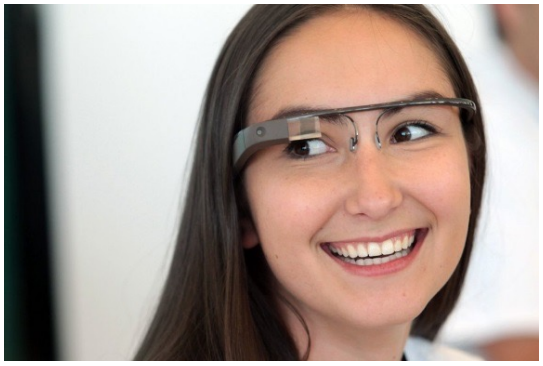
- **Two types of computing systems**
  - Servers, PCs, notebooks
  - Embedded systems
- **The next frontier**
  - Mainframe computing (60's-70's)
    - Large computers to execute big data processing applications
  - Desktop computing & Internet (80's-90's)
    - One computer at every desk to do business/personal activities
  - Embedded computing (21st Century)
    - "Invisible" part of the environment
    - Transformation of industry

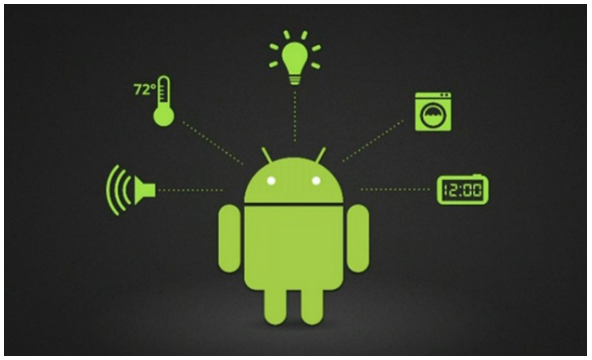


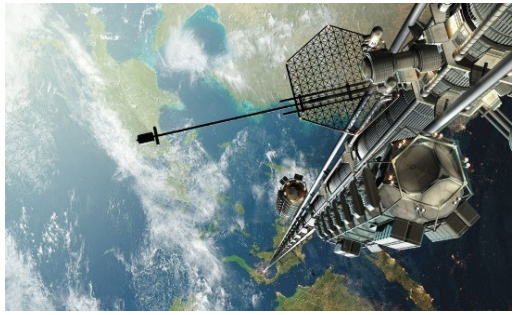
- **Number of microprocessor units per year**
  - Millions in desktops
  - Billions in embedded processors
- **Applications:**
  - Automotive Systems
    - Light and heavy automobiles, trucks, buses
  - Aerospace Systems
    - Airplanes, space systems
  - Consumer electronics
    - Mobile phones, office electronics, digital appliances
  - Health/Medical Equipment
    - Patient monitoring, MRI, infusion pumps, artificial organs
  - Industrial Automation
    - Supervisory Control and Data Acquisition (SCADA) systems for chemical and power plants
    - Manufacturing systems
  - Defense
    - Source of superiority in all weapon systems

Embedded Systems



















Hardware Components

- Powerful mobile processors

  -Trend: multi-processors within a single SOC

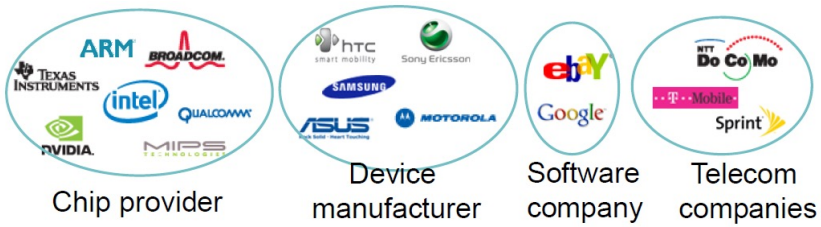
- Dedicated chips for display driver, touchscreen control, GPS, bluetooth, WiFi more…
- Storage/Memory
- Communications/Connectivity
- Various sensors

  -Accelerometer, proximity, gyro, temperature, etc.
- Camera(s)
- Battery

Mobile OS

- Apple iPhone OS
- Android for Googlephones
- Maemo OS for Nokia phones
- Blackberry RIM OS
- Windows CE (market share dropped from 23% in 2004 to 9% in 2009)
- Windows Phone
- **RTOS**, such as RT-Linux,VxWorks

.....



Types of Real-Time Systems

- Hard real-time system
  - —A system where “something very bad” happens if the deadline is not met
    - Examples: control systems for aircraft, nucluear reactors, chemical power plants, jet engines, etc.
- Soft real-time system
  - —A system where the performance is degraded below what is generally considered acceptable if the deadline is missed
    - Example: multimedia system

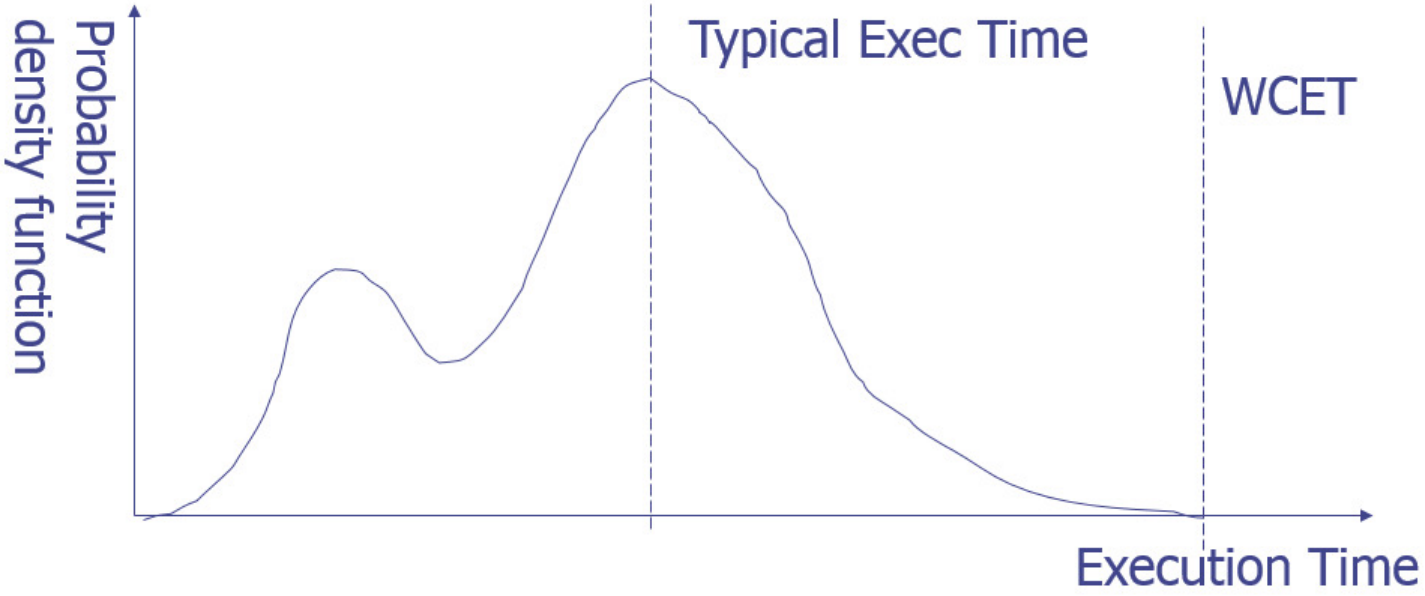
Time is Central to Embedded Systems

- Worst-case execution time (WCET) estimation
- Threshold property: can you produce a test case that causes a program to violate its deadline?
- Software-in-the-loop simulation: predict execution time of particular program path

ALL involve predicting an execution time property!

Worst-Case Execution Time (WCET) of a Task

- The longest time taken by a software task to execute->Function of input data and environment conditions
- BCET = Best-Case Execution Time (shortest time taken by the task to execute)





The WCET Problem

Given

- the code for a software task
- the platform (OS + hardware) that it will run on

Determine the WCET of the task.

- Why is this problem important?

  The WCET is central in the design of RT Systems: Needed for Correctness (does the task finish in time?) and Performance (find optimal schedule for tasks)



Programs as Graphs

```
1  #define EXP_BITS 32
2
3  typedef unsigned int UI;
4
5  UI modexp(UI base, UI exponent, UI mod) {
6    int i;
7    UI result = 1;
8
9    i = EXP_BITS;
10   while(i > 0) {
11     if ((exponent & 1) == 1) {
12       result = (result * base) % mod;
13     }
14     exponent >>= 1;
15     base = (base * base) % mod;
16     i--;
17   }
18   return result;
19 }
```

Basic Blocks

- A basic block is a sequence of consecutive program statements in which the flow of control enters only at the beginning of this sequence and leaves only at the end, without halting or the possibility of branching except at the end.

Example 15.2: The following three statements from the `modexp` function in Example 15.1 form a basic block:

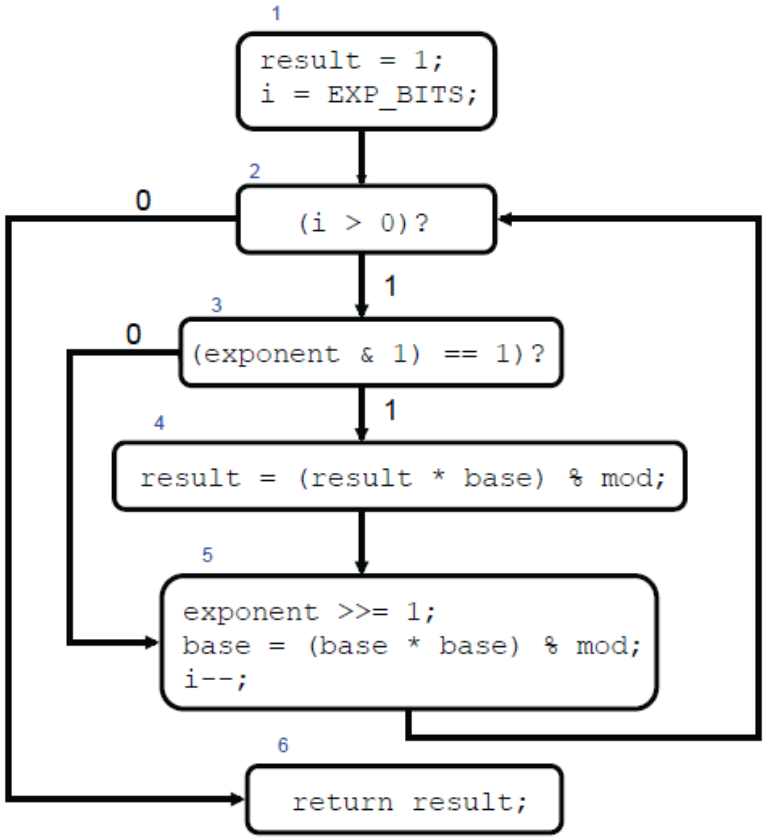
```
14    exponent >>= 1;
15    base = (base * base) % mod;
16    i--;
```

Another example of a basic block includes the initializations at the top of the function, comprising lines 7 and 9:

```
7   result = 1;
8
9   i = EXP_BITS;
```

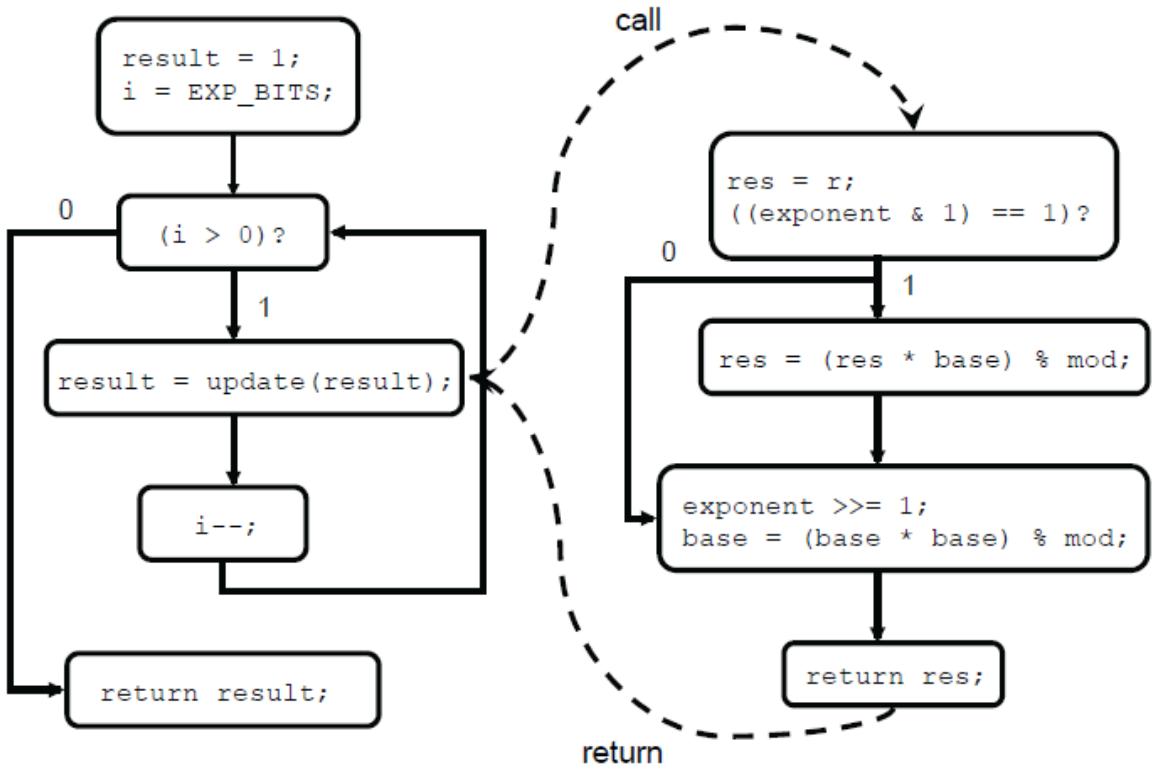
Control-Flow Graphs

- A control-flow graph (CFG) of a program P is a directed graph G = (V;E), where the set of vertices V comprises basic blocks of P, and the set of edges E indicates the flow of control between basic blocks.



Function Calls

```
1  #define EXP_BITS 32
2  typedef unsigned int UI;
3  UI exponent, base, mod;
4
5  UI update(UI r) {
6    UI res = r;
7    if ((exponent & 1) == 1) {
8      res = (res * base) % mod;
9    }
10   exponent >>= 1;
11   base = (base * base) % mod;
12   return res;
13 }
14
15 UI modexp_call() {
16   UI result = 1; int i;
17   i = EXP_BITS;
18   while(i > 0) {
19     result = update(result);
20     i--;
21   }
22   return result;
23 }
```





Factors Determining Execution Time

Program Path Analysis: Determining Loop Bounds

```
1  #define EXP_BITS 32
2
3  typedef unsigned int UI;
4
5  UI modexp(UI base, UI exponent, UI mod) {
6    int i;
7    UI result = 1;
8
9    i = EXP_BITS;
10   while(i > 0) {
11     if ((exponent & 1) == 1) {
12       result = (result * base) % mod;
13     }
14     exponent >>= 1;
15     base = (base * base) % mod;
16     i--;
17   }
18   return result;
19 }
```

Factors Determining Execution Time

Exponential Path Space

Example 15.6: Consider the function `count` listed below, which runs over a two-dimensional array, counting and accumulating non-negative and negative elements of the array separately.

```
1  #define MAXSIZE 100
2
3  int Array[MAXSIZE][MAXSIZE];
4  int Ptotal, Pcnt, Ntotal, Ncnt;
5  ...
6  void count() {
7    int Outer, Inner;
8    for (Outer = 0; Outer < MAXSIZE; Outer++) {
9      for (Inner = 0; Inner < MAXSIZE; Inner++)  {
10       if (Array[Outer][Inner] >= 0) {
11         Ptotal += Array[Outer][Inner];
12         Pcnt++;
13       } else {
14         Ntotal += Array[Outer][Inner];
15         Ncnt++;
16       }
17     }
18   }
19 }
```



Factors Determining Execution Time

Infeasible Paths

Example 15.7: Recall Example 12.3 of a software task from the open source Paparazzi unmanned aerial vehicle (UAV) project (Nemer et al., 2006):

```
1  #define PPRZ_MODE_AUTO2 2
2  #define PPRZ_MODE_HOME 3
3  #define VERTICAL_MODE_AUTO_ALT 3
4  #define CLIMB_MAX 1.0
5  ...
6  void altitude_control_task(void) {
7    if (pprz_mode == PPRZ_MODE_AUTO2
8        || pprz_mode == PPRZ_MODE_HOME) {
9      if (vertical_mode == VERTICAL_MODE_AUTO_ALT) {
10       float err = estimator_z - desired_altitude;
11       desired_climb
12             = pre_climb + altitude_pgain * err;
13       if (desired_climb < -CLIMB_MAX) {
14         desired_climb = -CLIMB_MAX;
15       }
16       if (desired_climb > CLIMB_MAX) {
17         desired_climb = CLIMB_MAX;
18       }
19     }
20   }
21 }
```



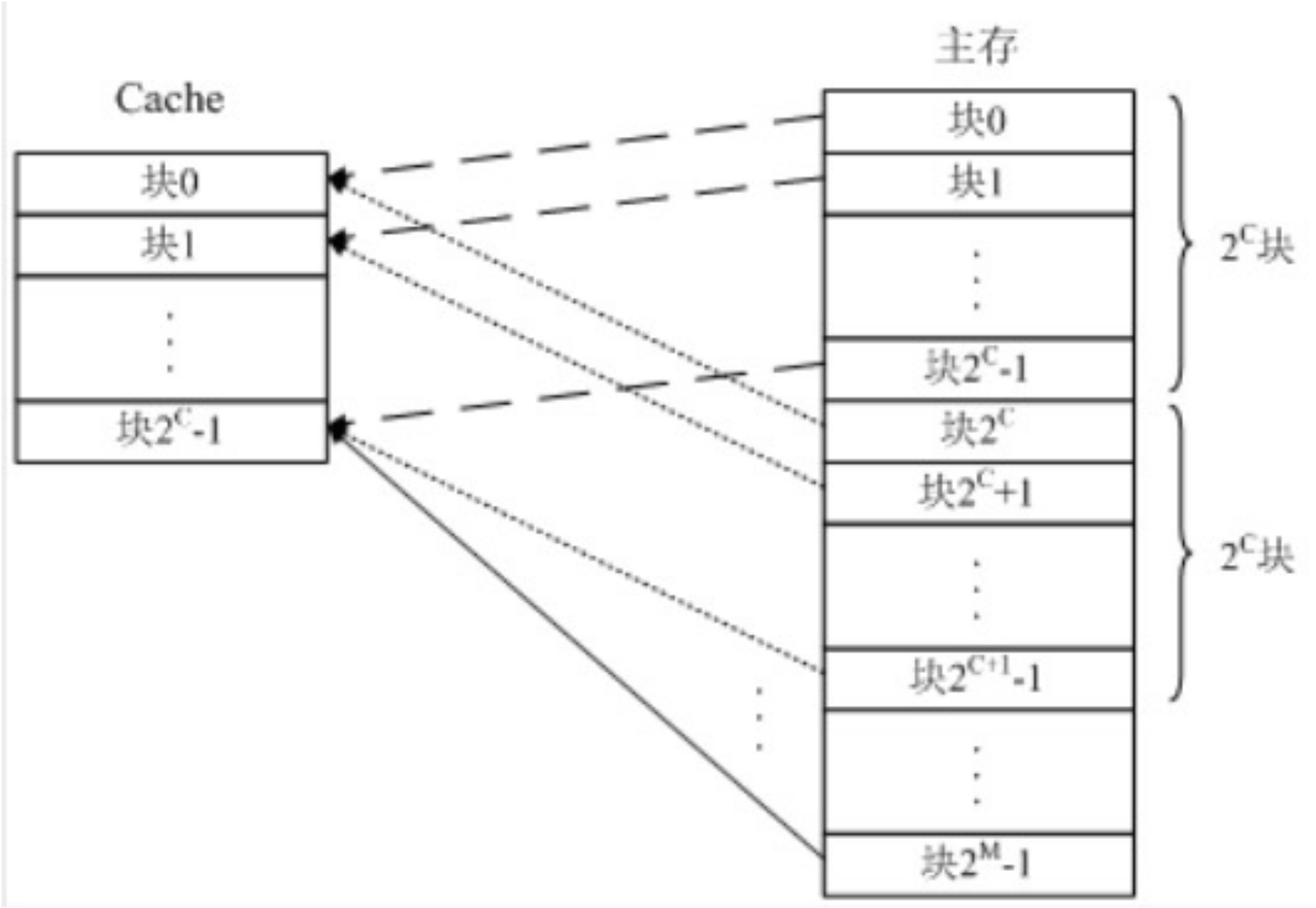
Processor Behavior Analysis: Cache Effects

```
1  float dot_product(float *x, float *y, int n) {
2    float result = 0.0;
3    int i;
4    for(i=0; i < n; i++) {
5      result += x[i] * y[i];
6    }
7    return result;
8  }
```

Suppose:

1. 32-bit processor
2. Direct-mapped cache holds two sets
   - 4 floats per set
   - x and y stored contiguously starting at address 0x0

What happens when **n=2**?


主存
Cache
块0
块1
块2^C-1
块0
块1
块2^C-1
块2^C
块2^C+1
块2^C+1-1
块2^M-1
2^C块
2^C块



Processor Behavior Analysis: Cache Effects

```
1  float dot_product(float *x, float *y, int n) {
2    float result = 0.0;
3    int i;
4    for(i=0; i < n; i++) {
5      result += x[i] * y[i];
6    }
7    return result;
8  }
```

Suppose:

1. 32-bit processor
2. Direct-mapped cache holds two sets
   - 4 floats per set
   - x and y stored contiguously starting at address 0x0

What happens when **n=8**?



Some WCET Estimation Tools

Commercial Tools: aiT, RapiTime, …

University/Research Tools: **GameTime**, Chronos, …

上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



第二部分：嵌入式系统测试方法介绍



目录

—快速回顾：软件测试典型技术

—嵌入式软件测试方法

—嵌入式软件测试策略

—嵌入式软件测试解决方案

软件测试技术——黑盒测试（1）

- 黑盒测试的概念
  - 若被测程序与特定的功能相联系，我们可以针对功能设计测试，以证实各功能完全可执行，同时在功能中寻找错误
  - 把测试对象看做一个黑盒子，完全不考虑程序内部的逻辑结构和内部特性，只依据程序的需求规格说明书，检查程序的功能是否正确。
  - 黑盒测试又叫做功能测试或数据驱动测试。
  - 黑盒测试类似于中医：使用望、闻、问、切
- 黑盒测试的目的
  - 是否有错误的或遗漏了的功能?
  - 在接口上，输入能否正确地接受? 能否输出正确的结果?
  - 是否有数据结构错误或外部信息访问错误?
  - 是否考虑了软件的出错情况?

软件测试技术——黑盒测试（2）

- 黑盒测试的一个常用技巧——打桩（Stub）
  - 用在隔离测试中，用以消除其他程序对被测程序的影响
  - 当被测程序调用其子模块时，可以使用模拟法，此时被测程序并没有真正调用其他模块，而是从桩模块处得到一个所需的值。这种技术叫做打桩。
- 黑盒测试常用方法
  - 等价类测试
  - 边界测试
  - 等等

软件测试技术——黑盒测试（3）

- 常用黑盒测试方法——等价分割（测试）
  - —等价分割是一种典型的黑盒测试方法
  - —等价分割将输入输出范围分割成多个等价的区域，然后从每个等价区中选取少数有代表性的数据做为测试用例。所谓等价，是指：
    - 在同一分割区内程序的行为是等价的
    - 等价区之间的依赖性可以忽略
  - —输入和输出并非只是程序参数：
    - 外部数据
    - 时间
    - 执行顺序/记录
    - 状态

软件测试技术——黑盒测试（4）

- 常用黑盒测试方法——边界值分析（测试）

  —边界值分析也是一种黑盒测试方法，是对等价分割方法的补充

  —经验告诉我们，大量的错误是发生在输入或输出范围的边界上，而不是在输入范围的内部。因此针对各种边界情况设计测试用例，可以查出更多的错误

  —使用边界值分析方法设计测试用例，首先应确定边界情况。应当选取正好等于，刚刚大于，或刚刚小于边界的值做为测试数据，而不是选取等价类中的典型值或任意值做为测试数据

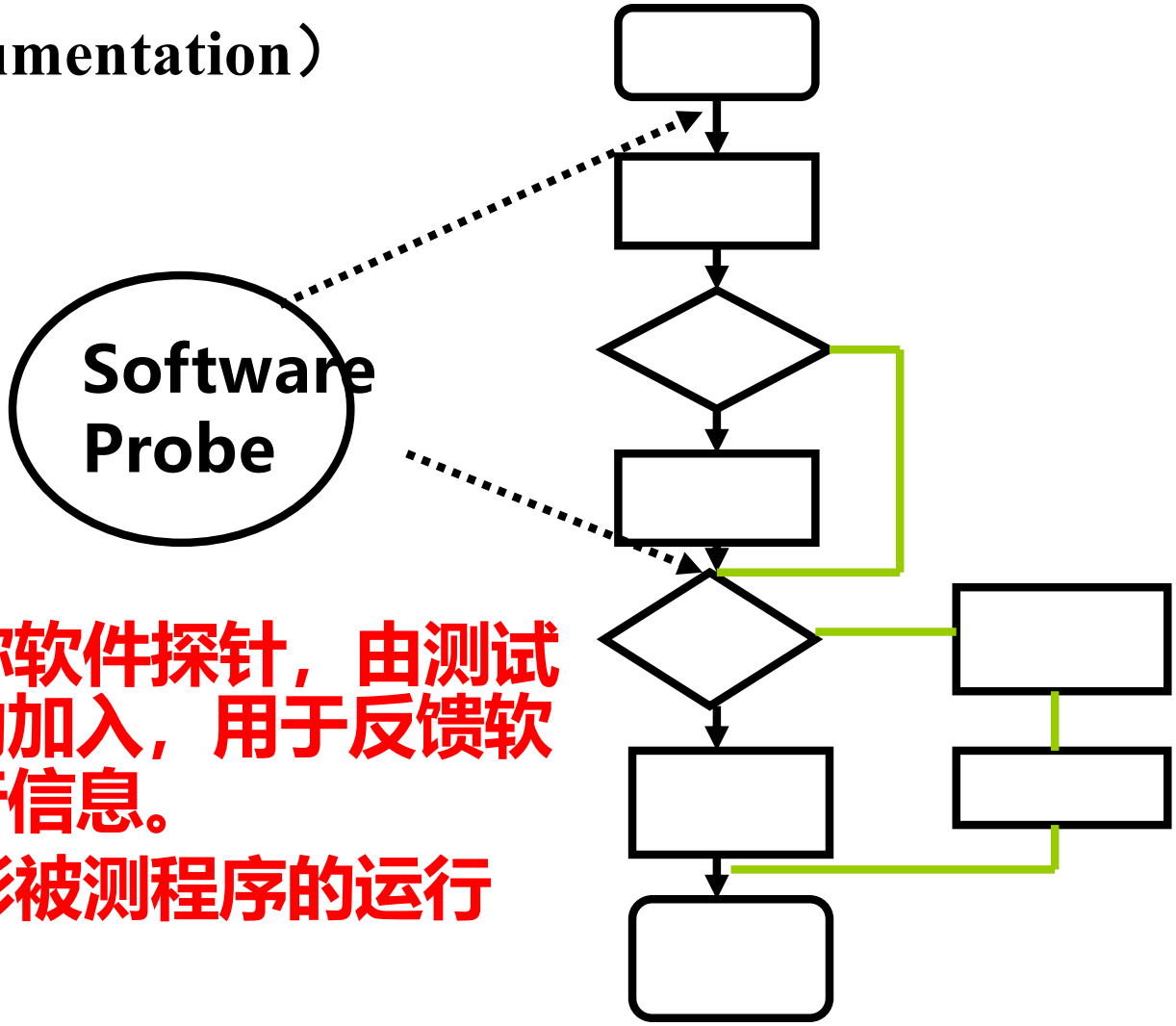
错误聚集在边界上，缺陷隐藏在角落里！

软件测试技术——白盒测试（1）

- 什么是白盒测试
  - —若被测程序与特定的结构相联系，我们可以针对结构设计测试，以确保内部的“所有齿轮相吻合”，即软件的内部过程是合理的，是遵照规定执行的
  - —白盒测试又称玻璃盒测试。把测试对象看作一个透明的盒子，充分利用其逻辑结构和有关信息，设计和选择测试用例。
  - —白盒测试又称为结构测试或逻辑驱动测试
  - —白盒测试类似于西医看病，通过X光、CT扫描等手段
- 白盒测试主要用于
  - —结构测试
  - —覆盖测试
  - —静态分析也可以算作白盒测试的一种手段

软件测试技术——白盒测试（2）

- 插桩（Instrumentation）



- 插桩又称软件探针，由测试工具自动加入，用于反馈软件的执行信息。
- 插桩不影被测程序的运行

软件测试技术——白盒测试（3）

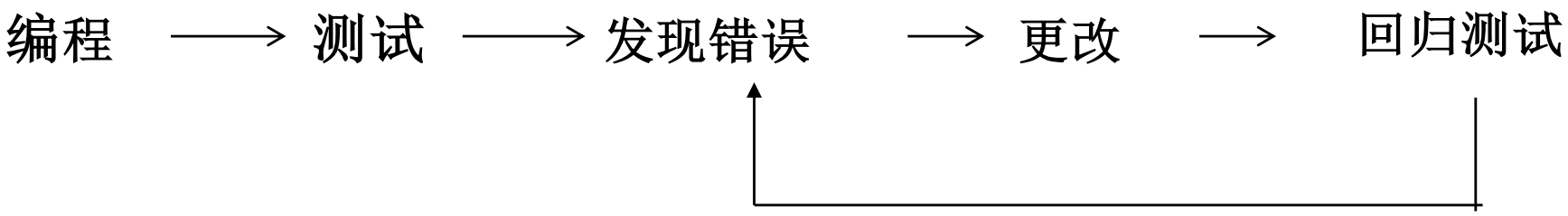
- 典型的白盒测试——基本路径测试
  - —Thomas McCabe首先提出
    - 从程序的结构入手，分析、计算模块的复杂程度——McCabe圈复杂度（=基本路径数）
    - 确认模块的一组基本路径，再根据这些基本路径设计测试用例，做到基本路径覆盖。
    - 圈复杂度可以通过程序流图得出
  - —基本路径测试是结构化测试的理论基础

软件测试技术——白盒测试（4）

- 代码覆盖
- 分支覆盖（判定覆盖）
- 条件覆盖
- 路径覆盖

软件测试技术——回归测试

- 回归测试
  - 软件更动后重新进行的测试
    - 既要测试变更部分,也要测试受影响部分
  - 关键在决定哪些测试必须被重复
  - 保证测试工作的可重现性
  - 尽量利用工具，提供回归测试的自动化水平



软件测试技术——静态分析（1）

- 静态分析又称静态测试

  - 静态测试是不动态执行程序代码而寻找程序代码中可能存在的错误或评估程序代码质量的过程。
  - 静态测试可以由人工进行，充分发挥人的逻辑思维优势。
  - 可以借助软件工具自动进行

软件测试技术——静态分析（2）

- 静态分析的主要方法和内容：

  —代码审查，这是软件质量保证（SQA）的重要内容

  —代码统计分析

  —软件工程度量
  - McCabe 复杂度
  - Halstead 复杂度
  - ...

嵌入式软件测试方法（1）

- 嵌入式软件测式的基本方法
  - —拿来主义——充分借用PC软件的测试方法
    - 静态/动态
    - 白盒/黑盒
    - 单元测试/集成测试/系统测试
    - ……
  - —全数字模拟测试
  - —交叉测试（Host/Target测试）
  - —真实环境验证

嵌入式软件测试方法（2）

- 全数字模拟测试
  - —采用数学平台的方法，将嵌入式软件从系统中剥离出来，通过开发CPU指令、常用芯片、I/O、中断、时钟等模拟器在HOST上实现嵌入式软件的测试
  - —主要特点
    - 与嵌入式硬件平台脱钩
    - 操作简单，可以借鉴常规的软件测试方法
    - 适用于功能测试
    - 有局限性

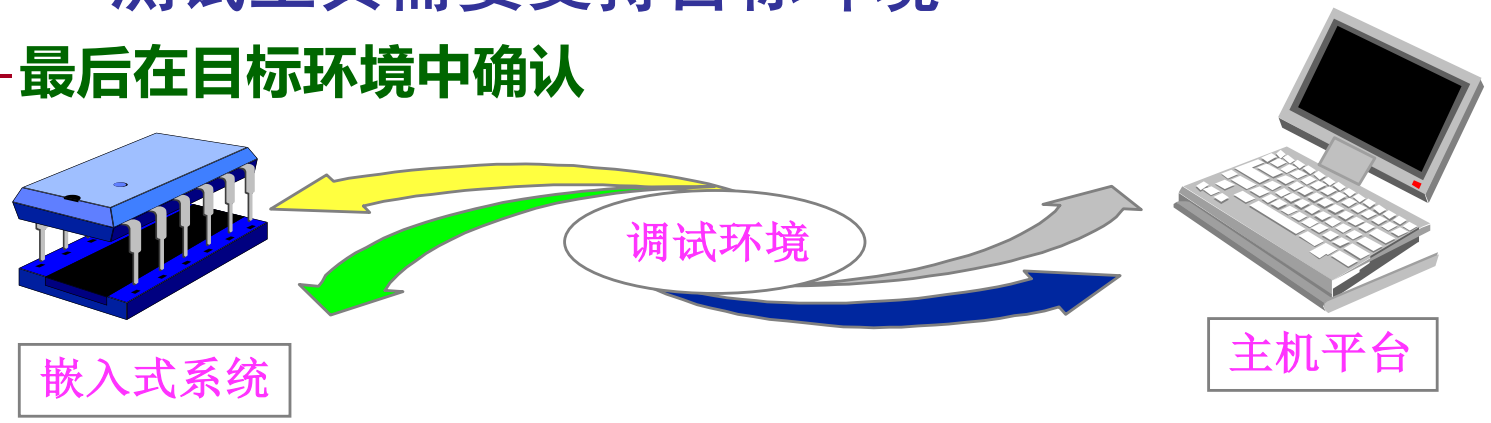


嵌入式软件测试方法（3）

- 全数字模拟测试的局限性
  - —通用性差，使用不同语言编写的嵌入式软件，需要不同的仿真程序来执行
  - —实时性与准确性难以反映出嵌入式软件的真实情况，测试出与时序有关的故障价值不大。
  - —维护统一、精确的系统时钟，理顺时序关系相当困难，特别是当并发的事件要求一定的同步关系时。
  - —开发成本高。设计出一个能进行系统测试的的环境代价太大
  - —只能作为嵌入式软件测试的辅助手段

嵌入式软件测试方法（4）

- 交叉测试（Host/Target测试）
  - —与目标环境无关的部分在PC机上完成
    - 充分利用高级语言的可移植性
    - 借鉴常规的软件测试方法
    - 与模拟测试不同
  - —与硬件密切相关的部分在Target上完成
    - 需要调试环境支持
    - 测试工具需要支持目标环境
  - —最后在目标环境中确认



嵌入式软件测试方法（5）

- 交叉测试的特点
  - —将大部分工作转移到PC平台上，在硬件环境未建好或调试工具缺乏时就可以开展
    - 可以充分地“拿来主义”
  - —适用于高级语言，如C，C++
  - —操作方便，测试成本较低。
  - —实时性受调试环境的制约
  - —目标环境中测试时要占用一定的目标资源
  - —注意目标环境和主机环境的差异：目标编译器的bug
  - —目标码的验证

嵌入式软件测试方法（6）

- 如何开展交叉测试（Host/Target测试）
  - 选用带有目标支持包(Target Package)的软件测试工具
  - 确定哪些模块与硬件无关，哪些与硬件相关
  - 配置相应的调试环境和目标环境
  - 分别进行Host和Target测试
    - Host：源代码+测试用例->编译连接->执行->测试结果
    - Target：源代码+测试用例+目标包->编译连接->下载->执行->反馈测试结果
- 交叉测试的嵌入式调试环境
  - —Simulator
  - —Emulator
    - BDM/JTAG Emulator

嵌入式软件测试方法（7）

- “拿来主义”
  - —静态分析很重要
    - Watts S. Humphrey的说法
      - 很多软件工程师认为动态测试比静态测试更重要——并非如此
      - 有经验的软件工程师平均每写1000行代码将会出现100个错误
      - 80%的软件错误归咎于对于编写语言的错误使用，而这些错误往往不是功能测试能解决的
      - 因此，软件工程师应该消除错误，找出根源，预防再次发生同样的问题
  - —静态分析的重要内容——代码规则检查
    - 实施简单、方便
      - 无需执行程序，与嵌入式环境无关
    - 早期介入，代价小，见效快
    - 有利于降低动态测试的难度
    - 有利于养成良好的编程习惯
    - 可以执行自定的规范

嵌入式软件测试方法（8）

- “拿来主义”

—动态测试不可少

动态测试是验证软件功能最直接、最有效的手段

- 通过运行被测程序验证其功能、性能，检查代码的执行情况
- 与静态分析相辅相成
- 需要事先设计详细、完备的测试用例
- 可用白盒、黑盒等方法
- 工作量较大、较枯燥

—动态测试的主要内容

- 功能、性能验证，是否符合需求定义
- 代码覆盖。哪些代码执行了，哪些没有执行，其比例如何

嵌入式软件测试方法（9）

- “拿来主义”
  - —白盒黑盒相辅成
    - 白盒测试与黑盒测试是软件测试最常用、最常规的两种技术
  - —白盒测试
    - 把测试对象看作一个透明的盒子，测试人员从其逻辑结构入手，设计和选择测试用例，对路径、控制结构、数据流等进行测试
    - 通过插装检查程序的状态，确定是否与预期的状态一致
    - 侧重于代码运行的过程
  - —黑盒测试
    - 把测试对象看做一个黑盒子，测试人员完全不考虑程序内部的逻辑结构，只依据其需求定义，检查程序运行的结果
    - 多用于功能测试和性能分析
    - 在程序的接口上进行
    - 需要设计“驱动”和“打桩”

嵌入式软件测试方法（10）

- “拿来主义”

—**单元集成两步走**

单元测试和集成测试是软件测试的两个阶段

—**单元测试**

- 将被测软件分解为单元，逐个测试
- 单元测试需要从程序的内部结构和功能出发设计测试用例。
- 多个模块可以平行地独立进行单元测试
- 可用白盒、黑盒等方法

—**集成测试**

- 在单元测试的基础上，将所有模块按照设计要求组装起来测试
- 主要测试内容
  - 接口间参数传递
  - 集成的功能实现
  - 模块间的影响

软件测试策略（1）

- 先静后动，从小到大，由黑到白
  - —先静态，后动态
    - 从代码规则检查做起
    - 测试开展得越早，付出的代价就越小
    - 静态分析简单、方便，成本低、见效快
    - 静态分析为动态测试打下良好基础
    - 大大降低了测试的成本
  - —先单元，后集成
    - ❑单元测试是集成测试的基础
    - ❑单元测试得越好，集成测试的工作量就越小
  - —先黑盒后白盒
    - ❑先验证软件功能是否满足需求
    - ❑后验证程序覆盖率，补充测试

嵌入式软件测试策略（2）

- 质量分析，事半功倍

通过改善代码的结构来分析、改进软件的质量——充分应用结构化测试技术

—软件模块的圈复杂度和逻辑结构能客观地反映软件的质量

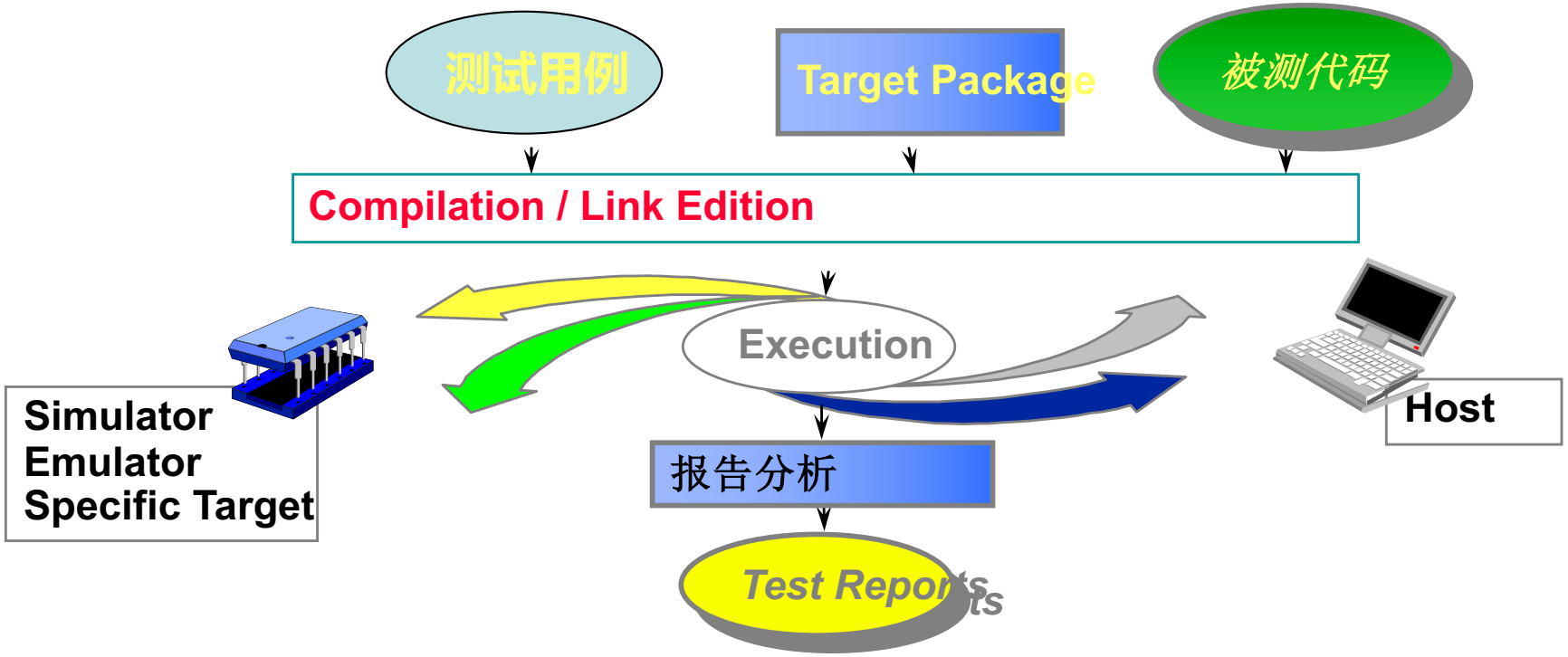
- **逻辑越“复杂”，就越容易出错。**
- **结构越“良好”，代码就越可靠**

—代码质量分析的好方法——结构化测试

- **从结构入手分析代码的复杂程度**
  - 逻辑复杂度定量化
  - 客观
  - 有理论基础
- **复杂度与代码出错的关联性非常强**
- **指导测试的执行**
- **指出代码质量改进的方向**

软件测试策略（3）

- 交叉测试，因地制宜
  ——嵌入式软件测试



嵌入式软件测试策略（4）

- 选好工具，抓好管理
  - 工欲善其事，必先利其器
  - 通常工具是必需的，但不是万能的
  - 根据测试需要和工具的特长进行选择
  - 使用测试工具带来的好处
    - 客观，准确，无感情色彩
    - 可长时间工作，不会疲劳
    - 高效、权威
    - 减轻人的工作量，但不能完全代替人
  - 测试需要管理
    - 测试要长期化、常态化、系统化
    - 测试需要维护、更新
    - 回归测试
    - 过程管理
    - 缺陷追踪



软件测试-代码质量

代码质量

CasePlayer
QA C/C++
Klocwork
Cantata++
Tessy
McCabe IQ

嵌入式常用软件测试工具介绍（1）

CasePlayer

— 面向嵌入式开发的程序规格说明书制作工具

— CasePlayer2是一种可以解析汇编、C语言（ANSI C或非ANSI C）的源代码，并且制作流程图以及模块规格说明书的制作工具



嵌入式常用软件测试工具介绍（2）

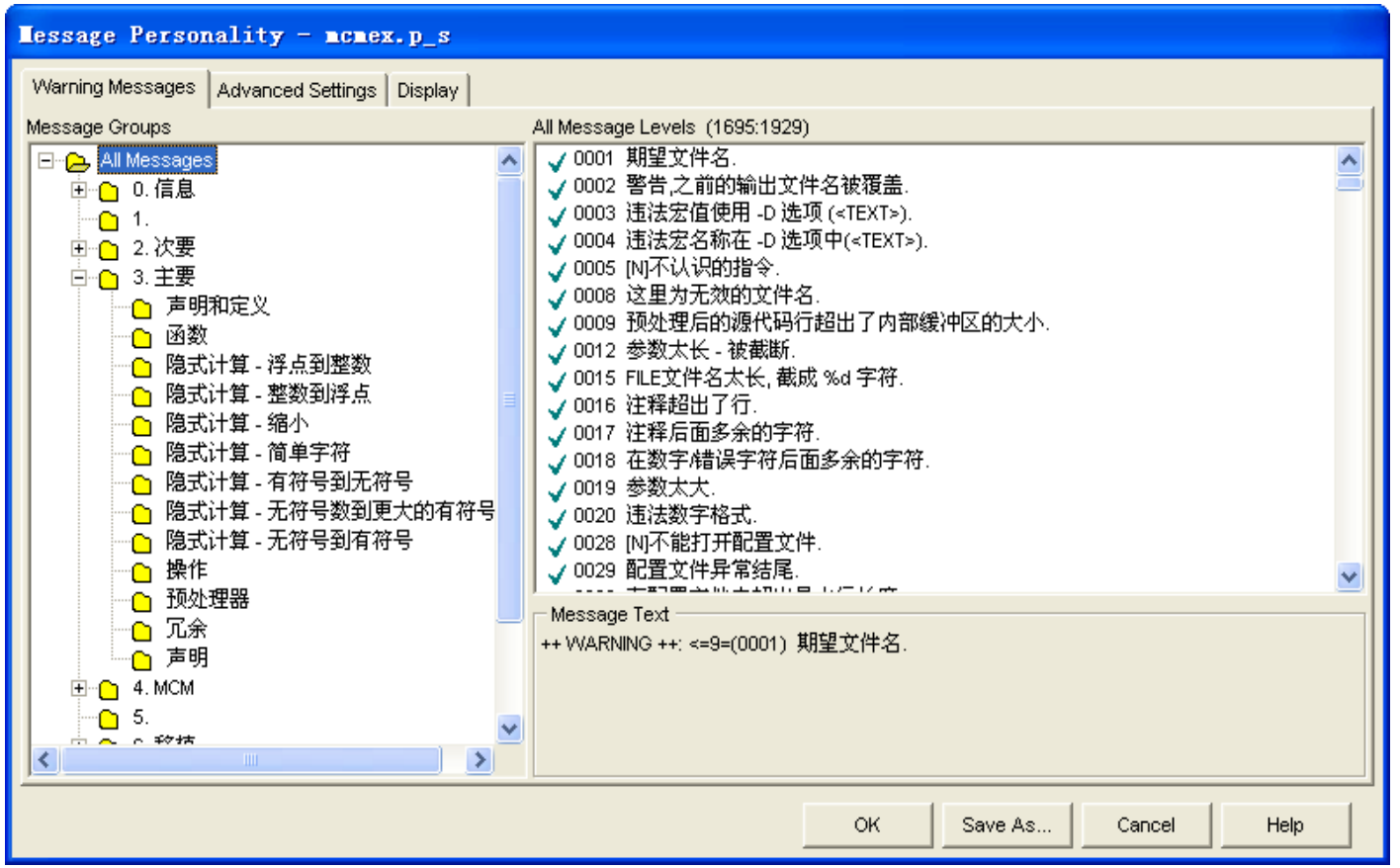
- PRQA——代码规则检查
  - —英国Programming Research公司出品
  - —代码规则检查
    - 分别针对C, C++, FORTRAN、JAVA
    - 支持MISRA C/C++等国际标准，及用户自定义的编程规范
  - —PRQA能发现
    - Ø 违背程序编写标准的问题
    - Ø 发现程序中不安全,不明确和模糊的部分
    - Ø 找出程序中不可移植部分
    - Ø 违背程序编译格式的问题
    - Ø 但不可以检测程序的实际功能的正确性
  - —PRQA还能
    - 提供多达44种业界广泛接受的度量
    - 提供多种可视化输出，包括函数结构图、函数调用树、外部参考、头文件关联、统计度量分析

嵌入式常用软件测试工具介绍（3）

QA C/C++/Java——代码规则检查

—规则可以选择、定制、汉化

—简单易用，见效快



嵌入式常用软件测试工具介绍（4）

- Tessy——自动单元测试
  - —公司:德国Hitex/Razorcat
  - —C/C++语言目标系统的单元测试（动态测试）
  - —分析源代码识别变量和类型，提供输入和输出的接口
  - —自动生成驱动和打桩
  - —管理测试用例，支持回归测试
  - —标准的测试报告
  - —测试驱动是基于Master/Slave结构，允许无限多的用例和很少的目标系统资源
  - —各种数据类型智能分析，包括指针的支持
  - —批处理功能可以在GUI和命令行下完成
  - —与嵌入式开发环境的无缝集成，占用资源很小
  - —支持MC/DC测试
  - —支持低端单片机

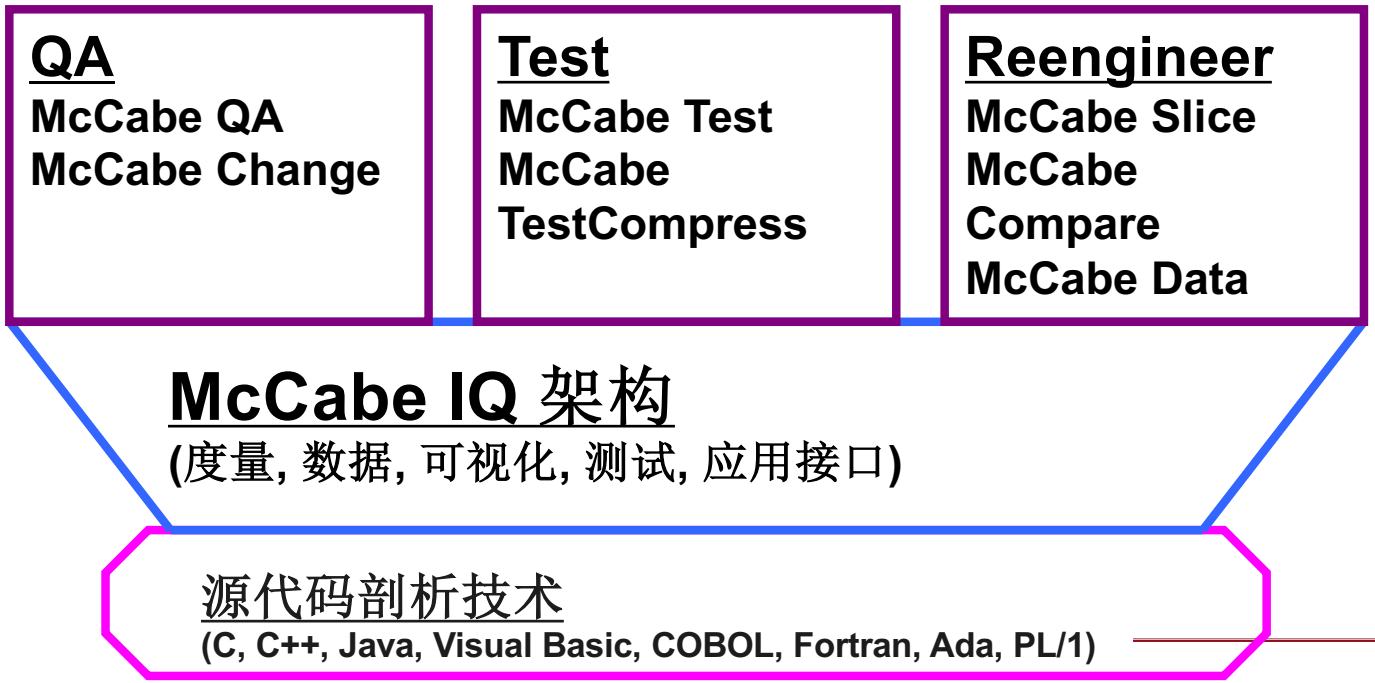
嵌入式常用软件测试工具介绍（5）

Tessy支持的嵌入式环境

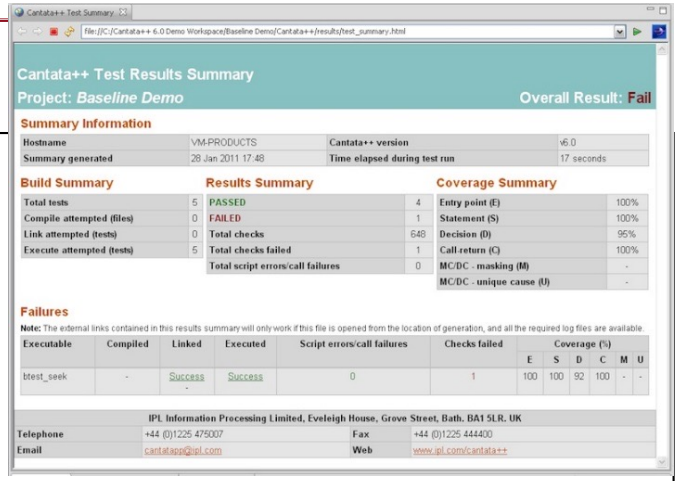
Compiler manufacturer	Architecture
ARM / ADS	ARM
Cosmic	68HC12, 68HC08, ST7
Fujitsu / FFMC16	MB90xxx
GNU ARM	ARM
GNU gcc	Host-PC
Greenhills	V850
HighTec	TriCore
IAR	68HC12, AVR, M16C, M32C, 78k0, V850
Keil	8051, C16x/ST10, XC166
Mentor Graphics / Microtec	68k
Metrowerks	68HC12, 68HC08, ST7
National	CR16
NEC	V850*
Tasking	C16x/ST10, TriCore, M16C*, XC166
Texas Instruments	TMS 470, TMS 320
Wind River Systems / Diab	PowerPC

嵌入式常用软件测试工具介绍（6）

- **McCabe IQ——结构化测试/质量分析**
  - —美国McCabe & Associates公司出品
  - —McCabe IQ适用于软件测试的不同阶段，主要构成：
    - McCabe QA（质量分析）
    - McCabe Test（追踪与测试）
    - McCabe Reengineering（再工程）。



嵌入式常用软件测试工具介绍（7）



- Cantata++——单元测试/集成测试
  - — 英国IPL公司出品
    - Cantata++ for C/C++
    - AdaTEST for Ada83/95
  - — 主要功能
    - *动态测试* *Does the code meet its specification ?*
    - *静态分析* *Does the code meet coding and complexity standards?*
    - *覆盖分析* *How much of the code been tested?*
  - — 主要特点
    - 自动脚本技术，自动生成驱动程序和桩模块
    - 独特的Wrapping技术，完全控制程序的接口，做到真正的逆向测试
    - 同时进行动态/静态/覆盖测试。用户可自定义通过/失败标准
    - 可支持中文界面
    - 支持嵌入式软件测试

嵌入式常用软件测试工具介绍（8）

Klocwork——软件缺陷检查

加拿大Klocwork公司出品

- 能够分析C、C++、C#和Java代码；
- 能够发现的软件缺陷种类：
  - 软件质量缺陷
  - 安全漏洞方面的缺陷
  - 对软件架构、编程缺陷的违反情况
- 对软件进行可视化的架构分析、优化
- 对软件进行各种度量分析
- 能够提供与多种主流IDE开发环境的集成
- 能够分析上千万代码行的超大型软件

